Department of Computer Science and Engineering

University of Ioannina

September 2025

INVENTORY OPTIMIZATION UNDER UNCERTAINTY: ADAPTIVE DECISION-MAKING WITH REINFORCEMENT LEARNING IN THE QUALITY-DEPENDENT NEWSVENDOR PROBLEM

Nefeli E. Sextou, MSc Candidate

Advisor: Konstantinos Parsopoulos, Professor

PRESENTATION OUTLINE

1. **Introduction**
2. **Problem Description**
3. **Reinforcement Learning Framework**
4. **Experimental Methodology**
5. **Experimental Results**
6. **Conclusion**

INTRODUCTION

INTRODUCTION

The present thesis is positioned at the intersection of **inventory theory** and **reinforcement learning**

Inventory theory:

- Optimizing supply-demand tradeoffs under uncertainty and real-world constraints
- Subfield of
  - Operations Research (OR)
  - Operations Management (OM)

Reinforcement learning:

- a type of *machine learning* where an agent learns by interacting with an environment and improving through feedback (rewards or penalties)

INTRODUCTION

Inventory Theory

Quality Dependent Newsvendor Problem (QDNP) by Özbilge *et al.* [1]:

- two-period newsvendor problem
- under demand uncertainty across the selling horizon
- incorporates product quality deterioration within demand modeling
- Incorporates donation

Operational Objective:

$$\underbrace{\textbf{Maximizing}\ \text{expected profit} + \text{Social and environmental responsibility}}_{\text{via } \textbf{donation}}$$

Role of donation:

- redirects surplus to food banks and charities
- contributes to tackling food waste
- improve profits via revenue loss offset:
  - government-backed corporate social responsibility (CSR) incentives such as tax deductions
  - appeals to socially conscious consumers
  - reduces disposal costs

INTRODUCTION

Reinforcement Learning

Soft Actor-Critic (SAC) by Haarnoja *et al.* [3]:

- **state-of-the-art**, outperforms other strong baseline RL algorithms on a large range of tasks (easy to complex)
- less sensitive to hyperparameters, easier to tune
- modeled to balance exploration and exploitation

Is RL employed on inventory problems?

- Boute et al. [4] note in their review on Deep Reinforcement Learning in Inventory Control that practical adoption is hampered by
  - the complexity of algorithm design
  - the heavy computational effort required
  - sensitivity in training

- These limitations however spark research interest
- RL is considered a promising but still maturing approach

INTRODUCTION

Why Reinforcement Learning (RL)?

- Two period structure → two sequential newsvendor problems → two sequential decision points

- RL can be situated within the broader stochastic optimization paradigm as a methodology for sequential decision-making under uncertainty
  - **Reinforcement Learning and Stochastic Optimization: A Unified Framework for Stochastic Optimization , W. Powell [2]**

- In RL, an agent learns to make **sequential decisions under uncertainty**

- Knowledge transfer

INTRODUCTION

The goal of this thesis was to try and answer the following questions:

1. Can SAC be applied on the QDNP as gradient free, stochastic optimization method, alternative to classic approaches, successfully ?

2. Does SAC show promising zero-shot transfer capabilities on the QDNP?

Real-world scenarios **don't guarantee prior knowledge of the distribution** of some factors
→ Strong interest in data-driven modeling

3. Can a variant of the model, which leverages kernel density estimation (KDE) to approximate the random demand of demand considered for the QDNP, perform adequately?

PROBLEM DESCRIPTION

QUALITY-DEPENDENT NEWSVENDOR PROBLEM (QDNP)

- Incorporates:
  - Product perishability
  - Consumer sensitivity to price and quality
  - Donation

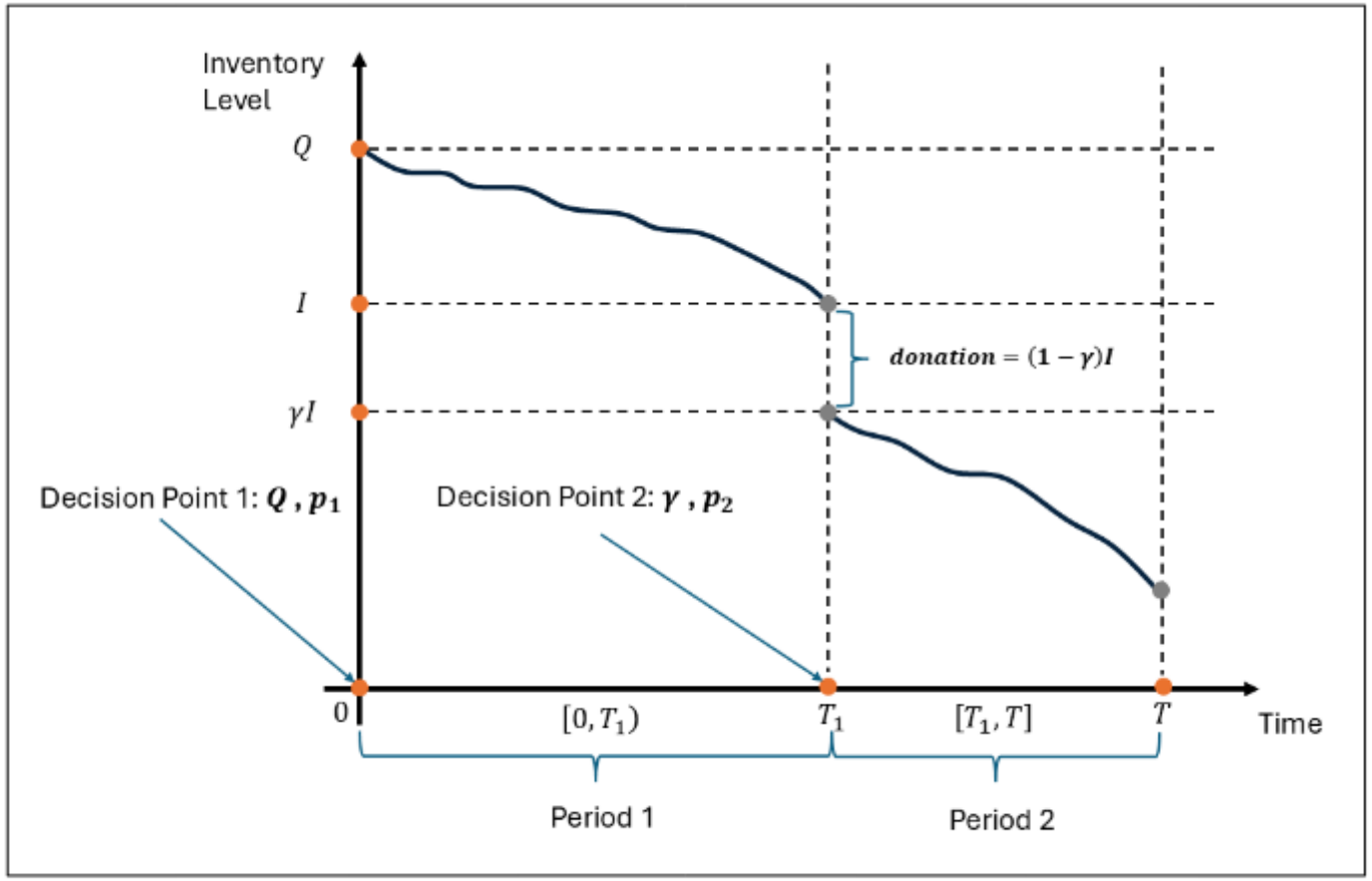
Distribution characteristics of $\boldsymbol{\varepsilon}$
- Non-decreasing hazard rate
- Finite mean μ, standard deviation $\sigma > 0$
- cdf $F(.)$: twice differentiable, invertible, and defined on an interval $[A, B]$

- **Real-world problem:** food retailer's pricing and inventory management problem where the product continuously deteriorates
- Unique batches of a fresh seasonal product
- 2-period, single-product
- Uncertain demand -> expressed thorough a random component of demand ε (follows the same distribution in both periods)
- Quality deterioration rates are assumed known: **time temperature indicator (TTI) data** + **food science models**
- Reward R>0 per donated product unit

Özbilge *et al.* [1]

QUALITY-DEPENDENT NEWSVENDOR PROBLEM (QDNP)

- $\boldsymbol{T}$: selling season $\boldsymbol{T} = \boldsymbol{T1} + \boldsymbol{T2}$
- $\boldsymbol{T1}$: update price and donation decision point
- $\boldsymbol{Period\ 1}$: length $\boldsymbol{T1 - 0} = \boldsymbol{T1}$
- $\boldsymbol{Period\ 2}$: length $\boldsymbol{T2} = \boldsymbol{T} - \boldsymbol{T1}$
- $\boldsymbol{Q}$: initial stocking quantity
- $\boldsymbol{I}$: inventory level at the end of Period 1
- $\boldsymbol{\gamma}$: percentage of retained inventory
- $(\boldsymbol{1} - \boldsymbol{\gamma})$: percentage of donated inventory
- $\boldsymbol{p_1}$: initial price
- $\boldsymbol{p_2}$: updated price for Period 2 (discounted)

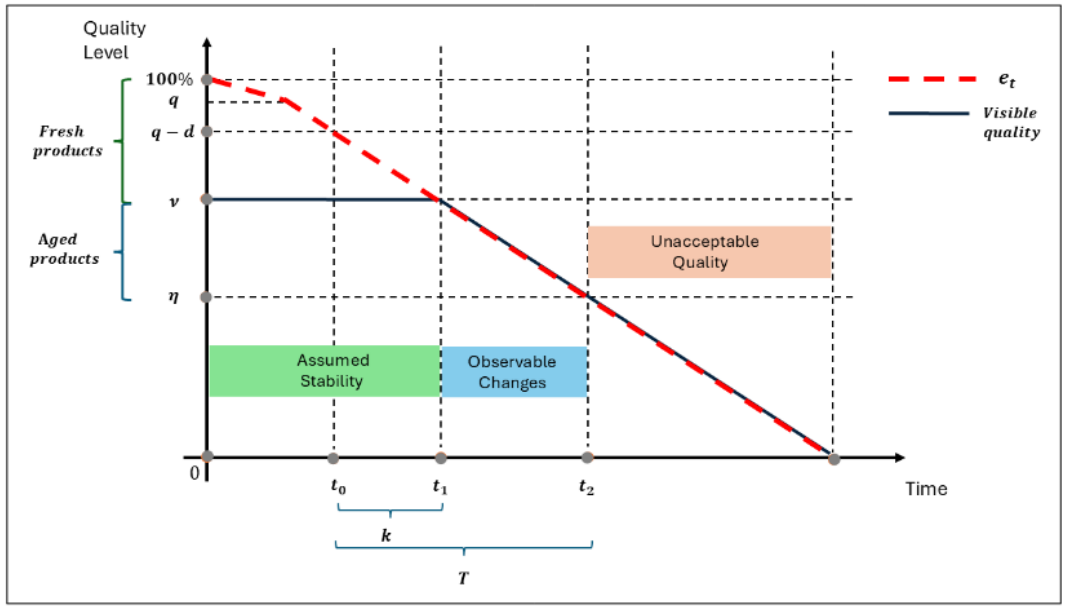


QDNP sequence of events

QDNP: QUALITY MODELING

Linear quality model:

$$\delta(t) = \begin{cases} \nu\,, & \text{if } t \leqslant \frac{q-d-\nu}{\lambda} \\ q-d-\lambda t\,, & \text{if } t > \frac{q-d-\nu}{\lambda} \end{cases}$$



Linear quality deterioration in the selling season

average case

$$\bar{\delta}_1(T_1) = \int_0^{T_1} \delta(t)\,dt$$
$$= \frac{1}{T_1}\left[\int_0^k \nu\,dt + \int_k^{T_1} (q-d-\lambda t)\,dt\right]$$
$$= \frac{1}{T_1}\left\{k\nu + \left[q-d-\frac{\lambda(T_1+k)}{2}\right](T_1-k)\right\}$$

$$\bar{\delta}_2(T_2) = \frac{1}{T_2}\int_{T_1}^{T} (q-d-\lambda t)\,dt = q-d-\frac{\lambda(2T-T_2)}{2}$$

$$k \equiv \frac{q-d-v}{\lambda}$$

$\boldsymbol{q}$: quality at supplier site
$\boldsymbol{d}$: quality drop in transportation
$\boldsymbol{v}$: discernibility threshold
$\boldsymbol{\eta}$: min acceptable quality
$\boldsymbol{\lambda} > \mathbf{0}$: deterioration rate at retailer site
$\boldsymbol{e_t}$: effective quality (from TTI data)

QDNP: DEMAND MODELING

- **Demand**
  - Is modeled as a function of:
    - Price
    - Time
    - Random demand component (uncertainty)

$$D_i(p_i, t, \varepsilon) = y_i(p_i, t) + \varepsilon, \quad i = 1, 2$$

- The **deterministic part of demand**:
  - Linear function
  - Decreasing in price
  - Increasing in perceived quality $\delta(t) \geq 0$
  - Modeling parameters:
    - $\boldsymbol{\alpha}$: describes **market size**
    - $\boldsymbol{\varphi} > \mathbf{0}$: describes **customers' sensitivity to quality**
    - $\boldsymbol{b} > \mathbf{0}$: describes **customers' sensitivity to price**

$$y_i(p_i, t) = \alpha + \phi\delta(t) - bp_i, \quad i = 1, 2$$

- Taking the **average case**:

$$\bar{y}_1(p_1, T_1) = \frac{1}{T_1}\int_0^{T_1} y_1(p_1, t)\, dt = a - bp_1 + \phi\,\bar{\delta}_1(T_1)$$

$$\bar{y}_2(p_2, T_2) = \frac{1}{T_2}\int_{T_1}^{T} y_2(p_2, t)\, dt = a - bp_2 + \phi\,\bar{\delta}_2(T_2)$$

QDNP: OBJECTIVE FUNCTION

- Total Expected Profit : $V = \mathbb{E}\left[\Pi_1(z_1, p_1)\right] + \alpha\, \mathbb{E}\left[\Pi_2(z_2, p_2)\right]$

- Stocking factors z_i for each period:
  - Measure the deviation between the supply rate and expected average demand
  - Central to modeling **shortage** and **surplus**

Stocking factors

$$z_1 = \frac{Q}{T_1} - \bar{y}_1(p_1, T_1)$$

$$z_2 = \frac{\gamma I}{T_2} - \bar{y}_2(p_2, T_2)$$

- Important mathematical properties for $\mathbb{E}[\Pi_i(z_i, p_i)]$
  - Concave w.r.t each decision variable z_i and p_i
  - V inherits concavity
    - Thus, each expected profit function can be expressed as a **single variable function**, of either decision variable
    - So, considering z_i the primary decision variable for each period, where p_i, replace:

$$a = 1$$

$$p_1(z_1) = \frac{a + \phi\bar{\delta}_1(T_1) + bC_0 + \mu}{2b} - \frac{\Theta(z_1)}{2b} \qquad p_2(z_2) = \frac{a + \phi\bar{\delta}_2(T_2) + bR + \mu}{2b} - \frac{\Theta(z_2)}{2b}$$

$\boldsymbol{C_0}$: purchasing cost per unit
$\boldsymbol{\Theta(z_i)}$: expected shortage

QDNP: OBJECTIVE FUNCTION

Period 1 Expected Profit:

$$\mathbb{E}\left[\Pi_1(z_1,p_1)\right] = \underbrace{(p_1 - C_0)\left[\bar{y}_1(p_1,T_1)+\mu\right]T_1}_{\text{marginal profit}} - \underbrace{C_0\,\Lambda(z_1)T_1}_{\text{surplus loss}} - \underbrace{(p_1 + C_s - C_0)\,\Theta(z_1)T_1}_{\text{shortage loss}}$$

Period 2 Expected Profit:

$$\mathbb{E}[\Pi_2(z_2,p_2)] = \left[\underbrace{(p_2 - R)\left(\bar{y}_2(p_2,T_2)+\mu\right)}_{\text{marginal profit}} - \underbrace{(R + C_d)\,\Lambda(z_2)}_{\text{surplus loss}} - \underbrace{(p_2 + C_s - R)\,\Theta(z_2)}_{\text{shortage loss}}\right]T_2 + \underbrace{RI}_{\text{reward earned if \textbf{all} inventory, available at T1 is donated}}$$

Expected Surplus:

$$\Lambda(z) = \int_A^z (z-u)\,\overset{\text{cdf}}{dF(u)} = \int_A^z (z-u)\,\overset{\text{pdf}}{f(u)}\,du$$

Expected Shortage:

$$\Theta(z) = \int_z^B (u-z)\,dF(u) = \int_z^B (u-z)f(u)\,du$$

Inventory level

$$\boxed{I = max(0,(z_1-\varepsilon)T_1)}$$

QDNP: OBJECTIVE FUNCTION

The maximization problem can be expressed as:

$$\max_{z_1, z_2} \quad V(z_1, z_2) \ ,$$

$$\text{s.t.} \quad \max\{A, \ -\overline{y}_2(p_2, T_2)\} \leqslant z_2 \leqslant \frac{I}{T_2} - \overline{y}_2(p_2, T_2)$$

PROPOSED NON-PARAMETRIC EXTENSION

- The **original** QDNP framework:
  - assumes **known and well-behaved demand shock distribution**
- **Real-world limitation**:
  - the **true distribution** of demand shock is often **unknown**

➢ Firms can collect historical demand shock observations over time, resulting in an ***empirical dataset***

- **Parametric distribution fitting**:
  - May impose a functional form that does not truly reflect ε
  - May not integrate well into an already existing model
- **Non-parametric approach for the QDNP**:
  - **Kernel density estimation (KDE)**
  - Addresses the limitation
  - Given a sufficiently *smooth* kernel:
    - ➢ **Preserves** ***smoothness*** and ***differentiability*** *properties* required by the QDNP

PROPOSED NON-PARAMETRIC EXTENSION

Kernel Density Estimation (KDE):

- A **non-parametric** method for estimating the probability density function of a random variable, based on observed data
- Builds a **smooth curve** that reflects the shape of the data
- Closely related to a histogram
  - With additional properties : **continuity + smoothness** (depends on kernel function)

Let $(x_1, x_2, \ldots, x_n)$ be a sample of independent and identically distributed observations drawn from an unknown continuous distribution with density $f(x)$

KDE of the density

$$\hat{f}_h(x) = \frac{1}{nh} \sum_{i=1}^{n} K\left(\frac{x - x_i}{h}\right)$$

$K(.)$: kernel function → determines the shape of local contributions around each data point
$h > 0$: bandwidth → regulates how wide each kernel is spread and controls smoothness

PROPOSED NON-PARAMETRIC EXTENSION

Integrating KDE into QDNP:

- Gaussian Kernel with mean $\boldsymbol{\mu} = \mathbf{0}$ and standard deviation $\boldsymbol{\sigma} = \mathbf{1}$
  - Smooth
  - Symmetrical
  - Infinitely differentiable
    - These properties are aligned with the analytical requirements of the QDNP

$$K(u) = \frac{1}{\sqrt{2\pi}} e^{-u^2/2}$$

- **Scott's rule** for bandwidth choice: $h = n^{-1/d+4}$

- Considering an empirical dataset $\boldsymbol{D}$ of the random demand shock $\boldsymbol{\varepsilon}$
  - Standard descriptive statistics are derived directly from the dataset
    - *Necessary* for the QDNP: $\boldsymbol{\mu}, \boldsymbol{\sigma}$
      - $[\boldsymbol{A}, \boldsymbol{B}] = [\boldsymbol{min}(\boldsymbol{D}), \boldsymbol{max}(\boldsymbol{D})]$
  - The expected **surplus** and **shortage** are now expressed as:
  - The cdf is: $\hat{F}(x) = \int_{-\infty}^{x} \hat{f}(t)\, dt$

$$\hat{\Lambda}_h(z) = \int_A^z (z-u)\, d\hat{F}_h(u) = \int_A^z (z-u)\hat{f}_h(u)\, du$$

$$\hat{\Theta}_h(z) = \int_z^B (u-z)\, d\hat{F}_h(u) = \int_z^B (u-z)\hat{f}_h(u)\, du$$

OPTIMIZING A QDNP WITH REINFORCEMENT LEARNING

SOFT ACTOR-CRITIC (SAC)

- **Off-policy:** the agent learns the best actions from past experiences, even if they were generated by a different strategy
- **Model-free:** the agent does not know how the environment works
- **Deep reinforcement learning (DRL):** use of neural networks
- **Continuous state and action spaces** (can support high dimensions)

Haarnoja *et al.* [3]

- **Stochastic Policy:** picks actions with certain probabilities
- **Main innovation:** augmentation of the standard RL objective with an **entropy term** (maximize reward + entropy)

$$J(\pi) = \sum_{t=0}^{\infty} \mathbb{E}_{(s_t,a_t)\sim T_\pi} \left[R(s_t, a_t) + \alpha \mathcal{H}(\pi(\cdot \mid s_t)) \right]$$

$\boldsymbol{a > 0}$: **temperature parameter**

*** or entropy coefficient**

$$\mathcal{H}(\pi(\cdot \mid s_t)) = -\mathbb{E}_{a_t\sim\pi} \left[\log \pi(a_t \mid s_t) \right]$$

Shannon entropy

- *Theoretical Foundation*:
  - Markov Decision Process (MDP) -based
  - Tabular form
  - Policy iteration + policy evaluation
  - Proven convergence to optimal policy

→ ***Neural networks*** → **large** *continuous* + *action* spaces

SOFT ACTOR-CRITIC (SAC)

Three Objectives:

- **Value Objective** : predicts how good a state is under the current policy by minimizing the regression loss between what the value network predicts and what is estimated through Q-values and the policy

$$J_V(\psi) = \mathbb{E}_{s_t \sim D}\left[\frac{1}{2}\left(V_\psi(s_t) - \mathbb{E}_{a_t \sim \pi_\phi}\left[Q_\theta(s_t, a_t) - \log \pi_\phi(a_t \mid s_t)\right]\right)^2\right]$$

- **Q-function Objective**: trains the Q-network to predict a "target" value derived by taking the immediate reward from the current state and adding the discounted predicted value of the next state as follows

$$J_Q(\theta) = \mathbb{E}_{(s_t, a_t) \sim D}\left[\frac{1}{2}(Q_\theta(s_t, a_t) - \hat{Q}(s_t, a_t))^2\right] \qquad \hat{Q}(s_t, a_t) = R(s_t, a_t) + \gamma\, \mathbb{E}_{s_{t+1} \sim T}\left[V_{\bar{\psi}}(s_{t+1})\right]$$

- **Policy Objective**: is related to the KL divergence between the policy distribution and a SoftMax distribution over Q-values

reparameterization trick

$$J_\pi(\phi) = \mathbb{E}_{s_t \sim D}\left[D_{\mathrm{KL}}\left(\pi_\phi(\cdot \mid s_t) \,\middle\|\, \frac{\exp(Q_\theta(s_t, \cdot))}{Z_\theta(s_t)}\right)\right] \xrightarrow{a_t \,=\, f_\phi(\epsilon_t; s_t)} J_\pi(\phi) = \mathbb{E}_{s_t \sim D, \epsilon_t \sim \mathcal{N}(0,I)}\left[\log \pi_\phi\big(f_\phi(\epsilon_t; s_t) \mid s_t\big) - Q_\theta\big(s_t, f_\phi(\epsilon_t; s_t)\big)\right]$$

  - The policy prefers actions that have both high Q-values and a low log-probability penalty, thus maintaining sufficient randomness in the process

SOFT ACTOR-CRITIC (SAC)

Algorithm 4.1 Soft Actor-Critic (SAC)

Initialize parameter vectors ϕ (policy), θ_1, θ_2 (Q-functions), and ψ (value function)
Initialize target parameters $\bar{\psi} \leftarrow \psi$
Initialize replay buffer $\mathcal{D}$
for each iteration **do**
  for each environment step **do**
    Sample action $a_t \sim \pi_\phi(\cdot \mid s_t)$
    Observe $s_{t+1} \sim p(\cdot \mid s_t, a_t)$ and reward $R(s_t, a_t)$
    Store new experience $(s_t, a_t, R(s_t, a_t), s_{t+1})$ in $\mathcal{D}$
  end for
  for each gradient step **do**
    Choose $min(Q(\theta_1), Q(\theta_2))$ for $\hat{\nabla}_\psi J_V(\psi)$
    $\psi \leftarrow \psi - \lambda_V \hat{\nabla}_\psi J_V(\psi)$ ▷ Value network update
    $\theta_i \leftarrow \theta_i - \lambda_Q \hat{\nabla}_{\theta_i} J_Q(\theta_i)$ for $i = 1, 2$ ▷ Q-network updates
    $\phi \leftarrow \phi - \lambda_\pi \hat{\nabla}_\phi J_\pi(\phi)$ ▷ Policy network update
    $\bar{\psi} \leftarrow \tau\psi + (1-\tau)\bar{\psi}$ ▷ Target value update
  end for
end for

avoid overestimation
due to off-policy $Q(s, a)$

scaled reward

$$R(s_t, a_t) = a^{-1} r(s_t, a_t)$$

SAC is sensitive to this!

PROPOSED ENVIRONMENT STRUCTURE FOR THE QDNP

- Analytical Solution: Backwards induction -> solve Period 2 first, then Period 1
- Episodic Structure: follow sequence of events -> **Two-step episode**
  - **Decision Variables:** z_1 and z_2
  - **First Step → apply action to z_1**
    - sample random component of demand and calculate the expected profit of *Period 1*
  - **Second Step → apply action to z_2**
    - calculate the expected profit of *Period 2*, after the agent perceives the consequences of uncertainty

State vector: $s_i = \begin{bmatrix} i, & I, & \varepsilon, & z_1, & z_2 \end{bmatrix}^{\mathsf{T}} \in \mathbb{R}^5$

Action vector: $a_i = \begin{bmatrix} z_1, & z_2 \end{bmatrix}^{\mathsf{T}} \in \mathbb{R}^2$

Rescaling to $[-1, 1]$: $\hat{x} = \dfrac{x - \mu}{s}$

μ: known or estimated empirically

s:half of the known feasible range

$\boldsymbol{i}$ is excluded from rescaling

PROPOSED ENVIRONMENT STRUCTURE FOR THE QDNP

Algorithm 4.2 Episode Structure for the QDNP Environment

```
Initialize step index i ← 0
Initialize state vector s_i ∈ ℝ^5
Initialize action bounds for a_i
Initialize episode reward V ← 0
for i = 0 to 1 do
    if i = 0 then                                   ▷ Step 0: First period
        Apply action a_0 to z_1: z_1 ← a_0
        Sample random demand component ε
        Estimate inventory level I = max(0, (z_1 − ε) T_1)
        Set reward for first step R_0 ← E[Π_1(z_1)]
        Update state vector s_0 with current step information
        Rescale state vector s_0 to [−1, 1]
        done = False                                ▷ Non-terminal step
        i ← 1                                       ▷ Prepare for next step
        return (s_0, R_0, done)
    else                                            ▷ Step 1: Second period
        Apply action a_1 to z_2: z_2 ← a_1
        Clip z_2 to feasible bounds
        Set reward for second step R_1 ← E[Π_2(z_2)]
        Update state vector s_1 with current step information
        Rescale state vector s_1 to [−1, 1]
        Compute total episode reward V ← R_0 + R_1
        done = True                                 ▷ Terminal step
        return (s_1, V, done)
    end if
end for
```

[A,B]

$$\text{s.t.} \quad \max\{A,\ -\overline{y}_2(p_2, T_2)\} \leqslant z_2 \leqslant \frac{I}{T_2} - \overline{y}_2(p_2, T_2)$$

EXPERIMENTAL METHODOLOGY

TEST DATA

Store	a	b	ϕ	q	d	v	h	λ/hr	T_1 (hrs)	T (hrs)	C_0	C_d	C_s	R
1	7.92	4.86	4.86	0.90	0	0.8	0.2	0.009	48	78	1	0.05	0.05	0.8
2	6.73	4.13	4.13	0.92	0	0.8	0.2	0.009	48	80	1	0.05	0.05	0.8
3	10.30	6.32	6.32	0.83	0	0.8	0.2	0.009	48	70	1	0.05	0.05	0.8
4	6.34	3.89	3.89	0.85	0	0.8	0.2	0.009	48	72	1	0.05	0.05	0.8

Parameters per problem instance

- Data used in the original study of Özbilge *et al.* [6]
- Parameters for **four supermarket branches** **(real-world modeling)**
- Product quality: **linearly degrading over time** **(vegetables)**
- The stochastic component of demand $\boldsymbol{\varepsilon}$ follows a **uniform** distribution with **zero mean**, and it is bounded in $[\boldsymbol{A}, \boldsymbol{B}] = [-\mathbf{2}, \mathbf{2}]$
- To **simulate** the distribution of $\boldsymbol{\varepsilon}$ being unknown: **produce empirical dataset**
  - Values of samples produced bounded in $[\boldsymbol{A}, \boldsymbol{B}] = [-\mathbf{2}, \mathbf{2}]$
  - $\boldsymbol{Data}$: 100000 samples
  - The implementation treats it as if the distribution is unknown -> $\boldsymbol{mean}(\boldsymbol{Data})$, $\boldsymbol{stdev}(\boldsymbol{Data})$, $[\boldsymbol{A}, \boldsymbol{B}] = [\boldsymbol{min}(\boldsymbol{Data}), \boldsymbol{max}(\boldsymbol{Data})]$ directly from the dataset, while **KDE estimates the pdf** wherever necessary

HYPERPARAMETERS

Hyperparameter	Value
Discount factor (γ)	0.99
Replay memory size	10^6
Number of hidden layers	2
Number of nodes per layer	256
Batch size	256
Time steps	46000
Target update interval	1

Fixed hyperparameter values

- Activation Function: **ReLU**
- Optimizer: **Adam**
- Episodes: **23000**
- Reward Scaling: **automatic**
  - ➢ *Stable-Baselines3 (Python)*

Label	Learning Rate	τ	Gradient Steps
H1	0.0003	0.005	1
H2	0.0003	0.005	4
H3	0.0003	0.05	1
H4	0.0003	0.05	4
H5	0.0003	1.0	1
H6	0.0003	1.0	4
H7	3e-06	0.005	1
H8	3e-06	0.005	4
H9	3e-06	0.05	1
H10	3e-06	0.05	4
H11	3e-06	1.0	1
H12	3e-06	1.0	4

Legend for hyperparameterization labels

- Based on hyperparameters presented in Haarnoja *et al.* [3]
- Other *StableBaselines3* hyperparameters : **default**
- Naming Conventions:
  - SAC applied on the "standard" QDNP → **SAC**
  - SAC applied on the KDE-enhanced QDNP → **SAC (KDE-QDNP)**

SAC EVALUATION

- "Deterministic" policy setting:
  - Mean of the Gaussian action distribution

- Same evaluation procedure was used for other variables: z_1, z_2, p_1, p_2, Q and γ

- Each data point is the mean return of 1000 samples per rollout
  - **Central Limit Theorem**
  - **Shapiro-Wilks** test:
    - Null-hypothesis: the data are normally distributed
  - If they **all pass** the Shapiro-Wilks test:
    - Apply **t-test** (pairwise)
  - Otherwise:
    - Non-parametric statistical testing

- Examined in conjunction with :
  - **Stability** and **output smoothness** during training:
    - **100-point moving average** of the total return
    - The **evolution of the entropy coefficient**
  - **Boxplots** and **descriptive statistics**

Algorithm 5.1 Policy Evaluation Procedure

```
Load trained SAC policy
Set policy to deterministic mode
for i = 1 to 100 do                                   ▷ evaluation rounds
    for j = 1 to 1000 do                              ▷ samples (rollouts)
        for t = 1 to 2 do                             ▷ steps in episode
            Generate action a_{j,t} = μ_φ(s_{j,t})
            Observe reward r_{j,t} and current state s_{j,t}
        end for
        Collect total episode reward R_j = r_{j,1} + r_{j,2}
    end for
    Compute mean total reward of round i: R_i = (1/1000) Σ_{j=1}^{1000} R_j
end for
Compute overall mean of the 100 round means: R_total = (1/100) Σ_{i=1}^{100} R_i
```

COMPARISON WITH ANALYTICAL AND MONTE CARLO BENCHMARKS

- Find best performing hyperparameterizations for both cases:
  - SAC and SAC (KDE-QDNP)

- Compare with:
  - **Analytical Solution** presented in Özbilge *et al.* [1]
  - **Monte-Carlo BFGS** benchmark (**MC-BFGS**)

Starting point (1,1) → reliable
- ➢ Difficult to find other reliable starting points

MC-BFGS requires using the full analytical formulation presented in Özbilge *et al.* [1]

Algorithm 5.2 Monte Carlo BFGS Maximization Procedure

```
Fix random seed for reproducibility
for i = 1 to 10000 do                                   ▷ Monte Carlo iterations
    Sample random demand component ε ~ U(A, B)
    Initialize candidate solution x_0 = (1.0, 1.0)
    Apply BFGS to maximize objective function V*(x_0; ε)
    while BFGS does not converge do
        Reinitialize x_0 with random values in [A, B]
        Retry BFGS
    end while
    Record optimized objective value V*
    Compute and store decision variables z_1, z_2, p_1(z_1), p_2(z_2), Q, γ
end for
```

ZERO-SHOT POLICY TRANSFER

- Train agent on the Store 1 problem instance
  - Directly apply to perturbed versions of the same instance without retraining
  - Compare with the outcome of retraining the model from scratch, with the perturbed setting (Ground Truth)

- Perturbations:
  - $\pm 10\%$, $\pm 20\%$, and $\pm 30\%$ of the original parameter values
  - Applied on a single parameter each time
  - The rest of the parameters remain fixed

a, b, and φ

demand modeling

C_0, C_d, C_s, and R

costs and reward

- Lazaric's Framework [5]
  - **Source task**: $M_{source} = (S, A, T_{source}, R_{source})$ → initial problem instance
  - **Target task**: $M_{target} = (S, A, T_{target}, R_{target})$ → each perturbed instance
  - Transfer with fixed $S \times A$
  - Transferred knowledge: policy π_{source} → applied to target tasks with no retraining

IMPLEMENTATION DETAILS

- Programming Language:
  - Python 3.10.12
- Libraries:
  - Stable-Baselines3 (RL)
  - Gymnasium (Environment Modeling)
  - NumPy
  - SciPy
  - Pandas
  - Matplotlib
- Seed, for reproducibility : 190

System	CPU Model	Threads/Core	Cores/Socket	RAM (GB)	Problem Instance
PC1	Intel(R) Core(TM) i7-4770 CPU @ 3.40GHz	2	4	8	1
PC2	Intel(R) Core(TM) i7-7700 CPU @ 3.60GHz	2	4	8	2
PC3	Intel(R) Core(TM) i7-2600 CPU @ 3.40GHz	2	4	8	3
PC4	Intel(R) Core(TM) i7-2600 CPU @ 3.40GHz	2	4	8	4

System characteristics and corresponding problem instances

MC-BFGS & Data Processing

- Jupyter Notebook
- 8GB RAM
- Intel(R) Core(TM) i7-8550U CPU @ 1.80GHz CPU
- 4 cores
- 8 threads

EXPERIMENTAL RESULTS

TUNING RESULTS

	SAC				SAC (KDE-QDNP)			
Problem Instance	**Store 1**	**Store 2**	**Store 3**	**Store 4**	**Store 1**	**Store 2**	**Store 3**	**Store 4**
Smooth (H)	1,2,3,5,6	1,3,5,6	1,3,5,6	1,3,4,5	1,2,3,5,6	1,2,3,5,6	1,3,5,6	1,3,4,5,6
Top 5 (based on Mean) (H)	7,11,8,5,6	3,10,1,9,2	9,3,4,7,10	9,3,7,5,10	5,8,11,4,6	10,2,9,7,3	3,4,10,7,8	7,3,8,12,2
Smooth & Top 5 (H)	5,6	3,1	3	3,5	5,6	2,3	3,5	3
Best (H)	5	3	3	3	5	3	3	3

Best hyperparameterizations per problem instance, for SAC and SAC (KDE-QDNP)

Problem Instance	Store 1	Store 2	Store 3	Store 4
Hyperparameterization	H5	H3	H3	H3
SAC	1038.21	798.66	6910.44	2502.00
SAC (KDE-QDNP)	1080.00	873.60	6949.44	2623.09

Runtimes (seconds)

- **Smooth**:
  - Plateau on 100-pt moving average of total return per episode
  - Smooth entropy coefficient from the exploration into the exploitation regime without oscillations } AND
    - **Stable bands** such as **between 0.2 and 0.4**, without abrupt jumps or drops
    - The agent is transitioning ***consistently from exploration to exploitation***
      - has learned a **sufficiently stable policy**
- **Top 5**:
  - Top 5 ranked hyperparameterization
  - Ranked by highest achieved mean (highest = rank 1)
- **Best**:
  - Statistical testing helps break ties

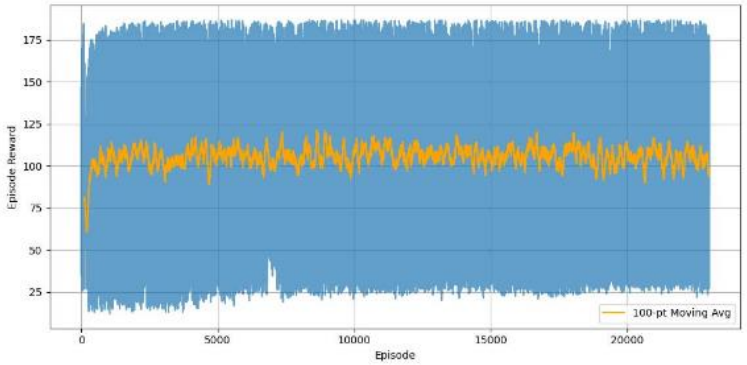
High value of a (near 1):
- **explorative** regime

Low value of a (near 0):
- **exploitative** regime

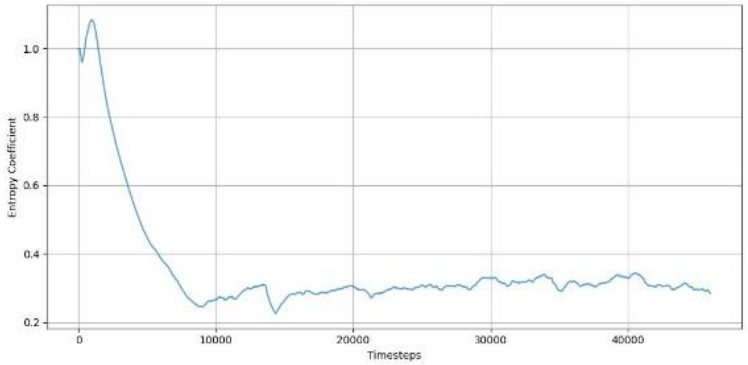
TUNING RESULTS

Example: Store 1, SAC

Acceptable :

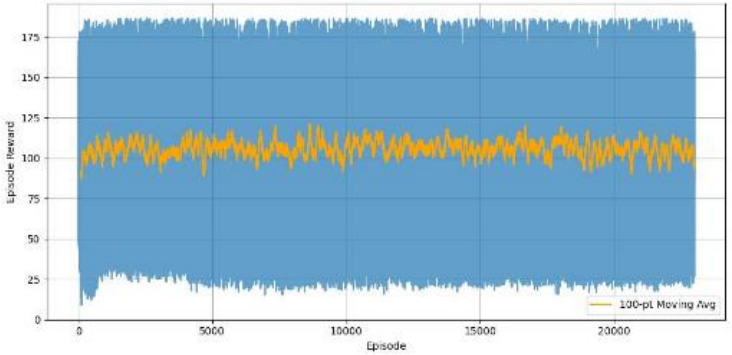


(c) Reward progress per episode (H3)

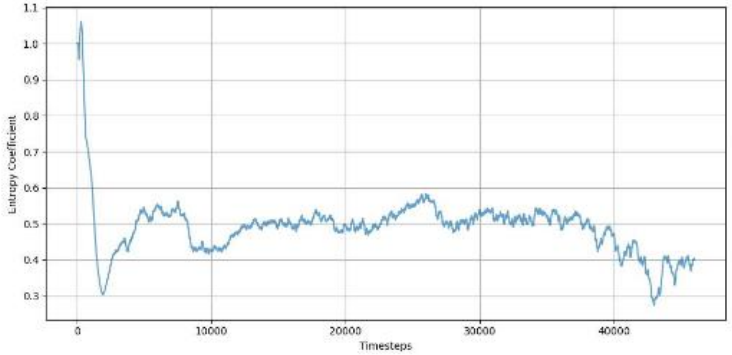


(d) Entropy coefficient per timestep (H3)

Not-acceptable:



(e) Reward progress per episode (H4)

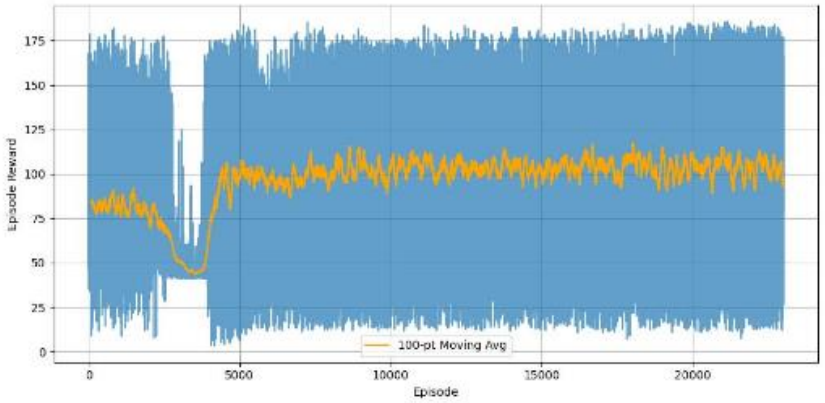


(f) Entropy coefficient per timestep (H4)

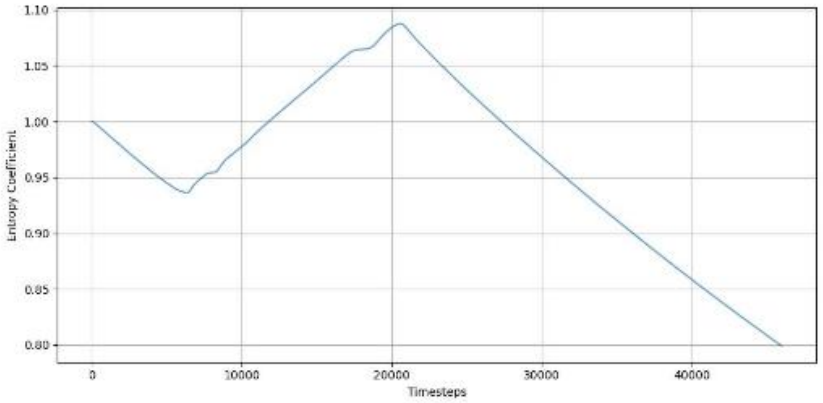
TUNING RESULTS

Example: Store 1, SAC

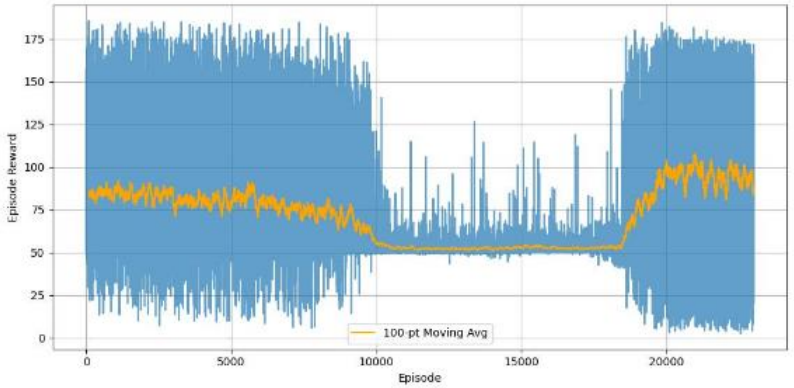
Not-acceptable (continued):



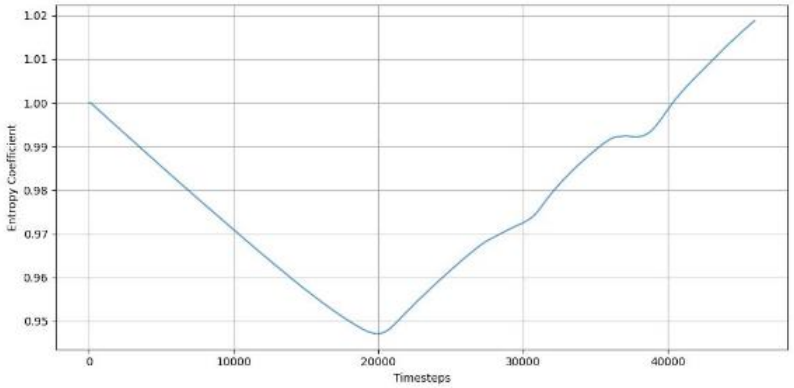
(e) Reward progress per episode (H8)



(f) Entropy coefficient per timestep (H8)



(g) Reward progress per episode (H9)



(h) Entropy coefficient per timestep (H9)

TUNING RESULTS

Example: Store 1, SAC

Hyperparameterization	H1	H2	H3	H4	H5	H6	H7	H8	H9	H10	H11	H12
Mean	94.914040	105.160780	94.931460	105.156053	105.871230	105.787896	105.900294	105.876871	94.904494	105.183404	105.896734	105.176055
Median	94.713588	104.955486	94.730840	104.948077	105.723738	105.622710	105.780687	105.751017	94.704251	104.977680	105.797274	104.970709
Std. Dev	1.580311	1.594083	1.580403	1.590601	1.595078	1.600612	1.587050	1.588341	1.580343	1.596268	1.582424	1.595093
Max	99.082860	109.372167	99.100522	109.358431	109.978907	109.943640	109.954304	109.946552	99.073303	109.399716	109.923573	109.390932
Min	91.191960	101.410906	91.208971	101.416251	102.099517	101.998907	102.191416	102.149159	91.182531	101.428708	102.247177	101.423698
Mean+Std. Dev	96.494351	106.754863	96.511863	106.746655	107.466308	107.388507	107.487344	107.465212	96.484838	106.779672	107.479159	106.771148
Mean-Std. Dev	93.333729	103.566696	93.351056	103.565452	104.276152	104.187284	104.313244	104.288530	93.324151	103.587136	104.314310	103.580962
Median+Std. Dev	96.293899	106.549569	96.311243	106.538678	107.318816	107.223321	107.367737	107.339357	96.284594	106.573949	107.379699	106.565802
Median-Std. Dev	93.133277	103.361402	93.150437	103.357475	104.128660	104.022098	104.193637	104.162676	93.123908	103.381412	104.214850	103.375616
Rank by Mean	11	8	10	9	4	5	1	3	12	6	2	7
Rank by Median	11	8	10	9	4	5	2	3	12	6	1	7

Descriptive statistics per hyperparameterization: Store 1 (SAC)

STATISTICAL TESTS AND BOXPLOTS

Example: Store 1, SAC

- When **p-value < 0.05**: statistical difference (mark with 1):

Otherwise: not enough evidence to reject the null hypothesis (mark with 0)

- They all **"pass"** the **Shapiro-Wilks** test:
  - Not enough evidence to say the data is not normally distributed
  - Assumed normally distributed

- **t-test**: helps find statistically different hyperparameterizations
  - **Finalists**: 5 and 6
  - t-test: **no statistical difference detected** (value = 0)

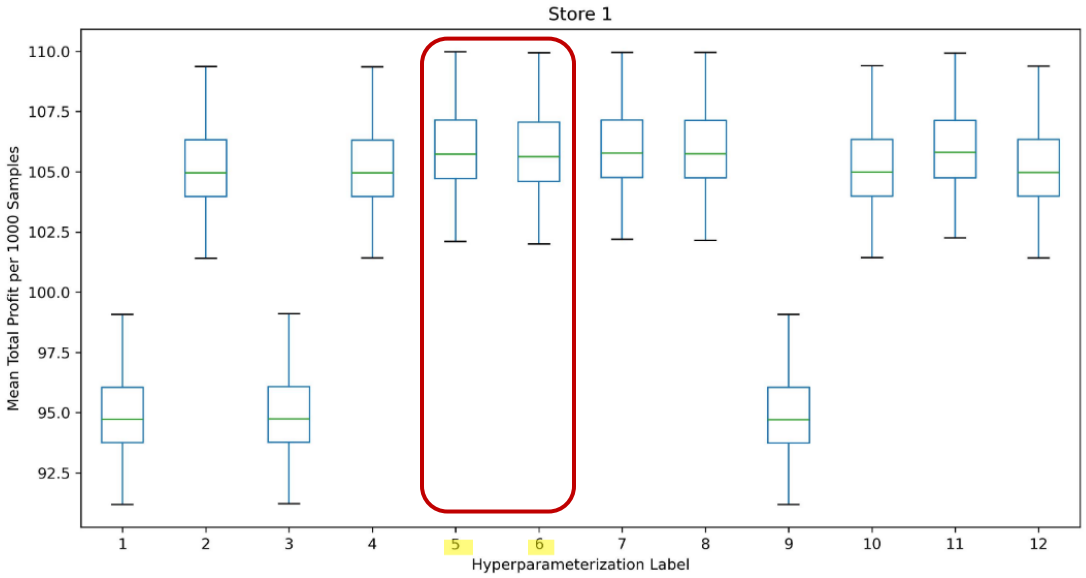
Store 1	H1	H2	H3	H4	H5	H6	H7	H8	H9	H10	H11	1H2
H1	-	1	0	1	1	1	1	1	0	1	1	1
H2	1	-	1	0	1	1	1	1	1	0	1	0
H3	0	1	-	1	1	1	1	1	0	1	1	1
H4	1	0	1	-	1	1	1	1	1	0	1	0
H5	1	1	1	1	-	0	0	0	1	1	0	1
H6	1	1	1	1	0	-	0	0	1	1	0	1
H7	1	1	1	1	0	0	-	0	1	1	0	1
H8	1	1	1	1	0	0	0	-	1	1	0	1
H9	0	1	0	1	1	1	1	1	-	1	1	1
H10	1	0	1	0	1	1	1	1	1	-	1	0
H11	1	1	1	1	0	0	0	0	1	1	-	1
H12	1	0	1	0	1	1	1	1	1	0	1	-

***t-test* results**

TUNING RESULTS

Example: Store 1, SAC

- Boxplots of H5 and H6 are almost identical
  - H5 median : 105.723738
  - H6 median: 105.622710
    - H5 is the best choice



Mean Return per hyperparameterization (Store 1)

"Significance Sums":

- Counts of statistical differences
  - The counts of 1 from the t-test result table

Problem Instance	H1	H2	H3	H4	H5	H6	H7	H8	H9	H10	H11	H12
Store 1	9	8	9	8	7	7	7	7	9	8	7	8

COMPARATIVE ANALYSIS

Method	Expected Profit	z_1	z_2	p_1	p_2	Q	γ	$1-\gamma$
Analytical	103.82	-	-	1.65	-	236.79	-	-
SAC	105.88	1.60	-1.08	1.65	1.25	229.17	0.85	0.15
SAC (KDE-QDNP)	108.62	1.67	-1.08	1.65	1.25	232.30	0.84	0.16
MC-BFGS	104.38	0.99	1.49	1.63	1.35	205.29	0.86	0.14

(a) Store 1

Method	Expected Profit	z_1	z_2	p_1	p_2	Q	γ	$1-\gamma$
Analytical	87.17	-	-	1.66	-	210.06	-	-
SAC	89.38	1.56	-0.99	1.66	1.24	206.07	0.82	0.18
SAC (KDE-QDNP)	92.17	1.51	-0.99	1.66	1.25	203.44	0.84	0.16
MC-BFGS	91.61	1.04	3.01	1.63	1.34	185.72	0.83	0.17

(b) Store 2

Method	Expected Profit	z_1	z_2	p_1	p_2	Q	γ	$1-\gamma$
Analytical	123.09	-	-	1.62	-	273.88	-	-
SAC	124.40	1.64	-1.27	1.62	1.25	267.40	0.82	0.18
SAC (KDE-QDNP)	126.41	1.95	-1.05	1.62	1.27	282.16	0.77	0.23
MC-BFGS	121.26	0.99	1.22	1.60	1.34	241.94	0.83	0.17

(c) Store 3

Method	Expected Profit	z_1	z_2	p_1	p_2	Q	γ	$1-\gamma$
Analytical	69.29	-	-	1.62	-	186.75	-	-
SAC	67.99	1.30	-0.98	1.61	1.21	179.98	0.73	0.27
SAC (KDE-QDNP)	70.42	1.27	-1.06	1.61	1.21	178.53	0.73	0.23
MC-BFGS	75.37	1.01	0.66	1.60	1.31	171.66	0.70	0.3

(d) Store 4

Aggregate (mean) results for all problem instances.

Instance	SAC	SAC (KDE)	MC-BFGS
Store 1	+2.06	+4.80	+0.56
Store 2	+2.21	+5.00	+4.44
Store 3	+1.31	+3.32	−1.83
Store 4	−1.30	+1.13	+6.08

(a) Expected profit differences

Instance	SAC	SAC (KDE)	MC-BFGS
Store 1	1.98%	4.62%	0.54%
Store 2	2.54%	5.74%	5.09%
Store 3	1.06%	2.70%	1.49%
Store 4	1.88%	1.63%	8.77%

(b) Relative error w.r.t the expected profit of the analytical solution

Instance	SAC	SAC (KDE)	MC-BFGS
Store 1	0.00	0.00	−0.03
Store 2	0.00	0.00	−0.03
Store 3	0.00	0.00	−0.02
Store 4	−0.01	−0.01	−0.02

(c) p_1 differences

Instance	SAC	SAC (KDE)	MC-BFGS
Store 1	−7.62	−4.49	−31.50
Store 2	−3.99	−6.62	−24.34
Store 3	−6.48	+8.28	−31.94
Store 4	−6.77	−8.22	−15.09

(d) Q differences

Comparisons with the analytical solution (Ground Truth).

- *Primary metric:* **Expected profit** (main objective)
- *Ground Truth* : **Analytical** solution
- *Accuracy Threshold*:
  - **around 5% -> extremely accurate**
  - **below 10% -> highly accurate**
  - **above 10% -> not accurate enough**
    - Based on Lewis forecast accuracy criterion for MAPE [4] :
      - Time periods are broken down into t ~ a time series

COMPARATIVE ANALYSIS

- SAC achieved consistent results, with relative errors no greater than **2.54%** of the expected profit (Store 2), when compared to the analytical solution
  - Errors are **negligible** in a practical decision-making setting
- SAC(KDE-QDNP) relative errors increased in all instances, except Store 4, with the largest difference being **5.75%** for Store 2
  - Errors are **negligible** in a practical decision-making setting
  - This corresponds to an average **misestimate** of approximately **0.06 per hour**
    - Given, $C_d = C_s = 0.05 \rightarrow \approx$ the value of 1 product unit per hour (a single salvage or disposal event)
- MC-BFGS yields inconsistent performance across problem instances
  - Store 4 relative error: **8.77%**
    - Closer to 10% : while still accurate, misestimates could be considered practically significant
  - High sensitivity to starting points
- A reported values were rounded to **2 decimal places**
  - For compatibility with the analytical solution results available
  - To reflect the level of precision used in managerial decision-making (i.e., supermarket prices)

ZERO-SHOT POLICY TRANSFER RESULTS

- *Source Task*: original Store 1 instance
- *Target Task*: perturbed Store 1 instance
- *Ground Truth*: retrained model with the perturbation
- *Primary Metric*: Expected profit
- *Perturbation:*
  - by percentage of the original parameter value

Results:

- *Very small relative errors in the majority of cases*
- *Only 2 scenarios that would **require retraining**:*

Store 1 - SAC on the standard QDNP

% of Original Value	a	b	ϕ	C_0	C_d	C_s	R
-10 %	0.16%	0.25%	0.02%	0.13%	0.47%	0.02%	0.12%
+10 %	0.30%	0.54%	0.17%	1.09%	0.01%	0.01%	0.30%
-20 %	4.86%	0.89%	0.08%	1.12%	0.01%	0.04%	1.01%
+20 %	0.74%	1.32%	0.02%	3.68%	0.05%	0.11%	1.63%
-30 %	16.09%	1.60%	0.12%	2.38%	0.03%	0.00%	1.76%
+30 %	1.49%	5.25%	0.21%	15.13%	0.04%	0.07%	4.30%

Expected profit relative error across parameter perturbations

Store 1 – Original Parameter Values

Store	a	b	ϕ	C_0	C_d	C_s	R
1	7.92	4.86	4.86	1	0.05	0.05	0.8

- ***Reliable transfer capabilities under partial model mismatch – robust source task policy behavior***
- ***High potential for SAC in online systems capable of accurate, real-time decision-making***

CONCLUSIONS

CONCLUSIONS

- **SAC** reliably approximates the analytical solution
  - Small errors
  - Across multiple real-world problem instances
  - Remaining well within practical tolerances for decision-making

- **KDE-enhanced model** introduced slight increases in approximation error but remained within the same tolerance level

- **Zero-shot policy transfer** experiments showed that a pretrained SAC policy can generalize effectively across changes in demand and cost parameters
  - **Small and medium** perturbations → **negligible deviations** -> **avoids retraining**
  - **Slightly larger** perturbations → **retraining** for select parameters

- Adds to the discussion of how RL can be applied in OM:
  - SAC can provide a stable and effective way of solving the QDNP and its data-driven variant
  - The considered setting is a useful testbed:
    - **Analytical benchmark** for comparison
    - Contains **sequential decision-making embedded** in its formulation
    - The **stochastic component** is **isolated** in a **controlled** and **testable** way

CONCLUSIONS

- RL:
  - Highly potent and flexible tool for firms seeking:
    - Adaptive and data-driven inventory management strategies
    - Reduction in reliance on frequent retraining or rigid analytical models

- The present thesis provides a focused example of RL:
  - Well-structured inventory model
    - Lays the groundwork for extending such methods to more complex and data-driven decision problems

- Future work:
  - Explore larger and more diverse set of store instances
  - Explore alternative quality deterioration patterns
  - Consider a wider range of demand distributions
  - Evaluate policy transfer under simultaneous changes in multiple parameters
  - Integrating more data-driven components

THANK YOU!

MENTIONED SOURCES

- **[1]** A. Özbilge, E. Hassini, and M. Parlar, “Optimal pricing and donation policy for fresh goods,” European Journal of Operational Research, vol. 312, no. 1, pp. 198–210, 2024. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0377221723004708

- **[2]** W. B. Powell, Reinforcement Learning and Stochastic Optimization: A Unified Framework for Sequential Decisions. Hoboken, NJ: John Wiley & Sons, Inc., 2022. [Online]. Available: https://doi.org/10.1002/9781119815068

- **[3]** T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, “Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor,” 2018. [Online]. Available: https://arxiv.org/abs/1801.01290

- **[4]** R. N. Boute, J. Gijsbrechts, W. van Jaarsveld, and N. Vanvuchelen, “Deep reinforcement learning for inventory control: A roadmap,” European Journal of Operational Research, vol. 298, no. 2, pp. 401–412, 2022. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0377221721006111

- **[5]** A. Lazaric, Transfer in Reinforcement Learning: A Framework and a Survey. Berlin,Heidelberg: Springer Berlin Heidelberg, 2012, pp. 143–173. [Online]. Available:https://doi.org/10.1007/978-3-642-27645-3_5

- **[6]** C. D. Lewis, Industrial and business forecasting methods : a practical guide to exponential smoothing and curve fitting. London ;: Butterworth Scientific, 1982.